# MemForge — Understanding the Project, and the Runbook to Finish It

An inference engine built from scratch: paged KV cache, continuous batching, prefix sharing and speculative decoding — and the runbook to finish measuring it.

---

**Repository** memforge · branch `main`

**Measured on** 2026-10-26

**Hardware** Intel 10-core / 23.7 GB RAM · NVIDIA RTX 4050 Laptop, 6141 MiB, compute 8.9

**Stack** torch 2.11.0+cu128 (CUDA 12.8) · Python 3.12.7 · Windows 11

**Test state** 398 Python tests · 18 C++ cases · lint clean · 0 blocks leaked, 0 memory errors

**Written for: the next person to sit down at this repo and finish Phase E and D.** That might be you in a month. It assumes you can read Python and C++ but know nothing about this codebase, and nothing about why any of its decisions were made.

> **On the numbers in this document.** Every figure comes from a committed result in `benchmarks/results/` on the hardware named above, and every chart is generated from those files rather than transcribed. Where a result is unfavourable it is stated as such — two of the features measured here are losses on this hardware, and both say so.

Everything here was measured on this machine on **2026-10-26**. Where a number is unfavourable, it says so — a report that only contains wins is not a report.

---

## 1. What this project actually is

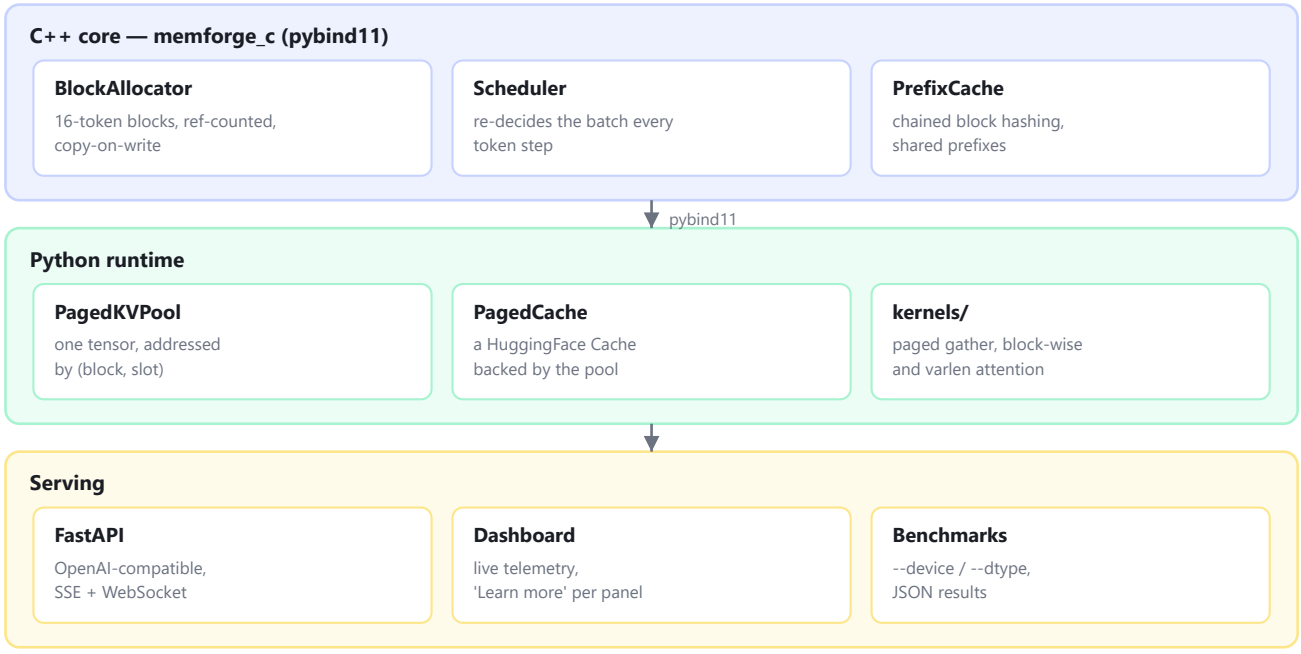



Figure 1 — The three layers. Memory management is C++; orchestration is Python; pybind11 is the only seam.

MemForge is an **LLM inference engine written from scratch**. Not a wrapper around vLLM — the allocator, the scheduler and the attention kernels are all here. The point is to implement the four ideas that make modern LLM serving fast, and to *prove* each one works rather than assume it.




## The four ideas

1. **PagedAttention.** A sequence's KV cache lives in scattered fixed-size blocks instead of one contiguous slab. Contiguous allocation has to reserve for the longest output a request *might* produce; almost every request stops earlier, and that unused tail is pure waste. Blocks are handed out as tokens are actually generated, so waste is at most one partly-filled block per sequence. Fragmentation is 0% by construction: every block is the same size, so a freed block always fits the next request.
2. **Continuous batching.** The scheduler re-decides the batch every token step. A finished request leaves immediately; a new one joins mid-flight. There is no static batch to wait for. The metric is `tokens_per_forward_pass`: 1.0 means nothing is batched, and a forward pass is the expensive thing because it reads every weight.
3. **Prefix sharing.** Two requests starting with the same tokens point at the same physical blocks. The hash is **chained** — each block's hash mixes in the previous block's — because a block's KV depends on *all* preceding tokens through attention. Hashing blocks independently would match a block computed from a different history and serve another conversation's KV.
4. **Speculative decoding.** A small draft model guesses K tokens; the big model checks all K in one pass. Rejection sampling against the target's own distribution makes the output *identical* to what the target alone would produce — it is a scheduling trick that cannot change the answer.

## The one rule that explains the whole test suite

**A wrong KV read does not crash. It produces fluent, confident, wrong text.**

That is why correctness here is asserted as *byte-identical output*, not "close enough", and why nearly every optimisation has a test comparing its tokens against an unoptimised path. It is also why the tests are **mutation-tested**: deliberate bugs are injected to confirm the tests actually fail. That found several tests which passed against broken code — including an entire acceptance path no test was exercising.

---

## 2. Where the project stands

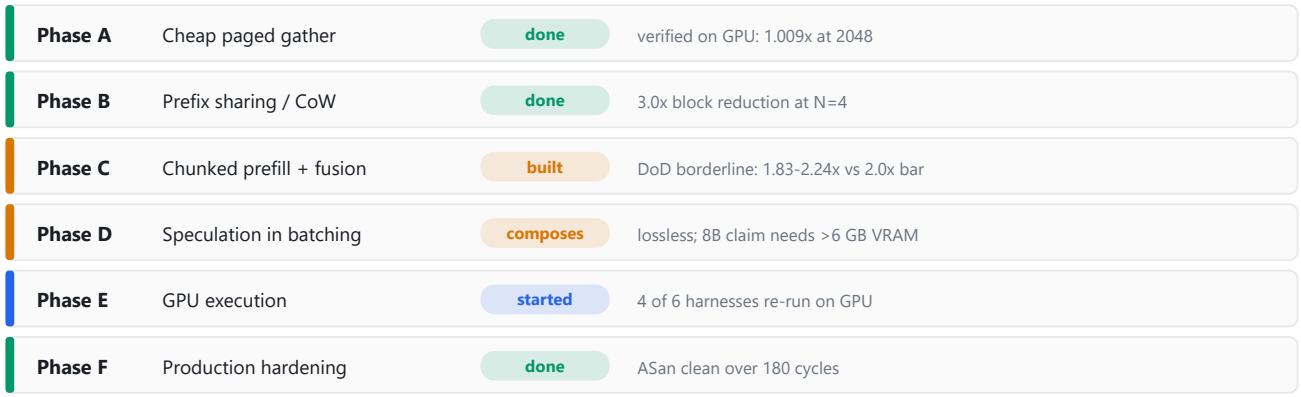



Figure 2 — Phase status. Amber means built and working but with a definition of done written for hardware this machine does not have.

| Phase | What it is | Status |
|---|---|---|
| **A** | Make the paged gather cheap | ✅ done — and **verified on GPU** |
| **B** | Prefix sharing via copy-on-write | ✅ done — verified on GPU |
| **C** | Chunked prefill + prefill/decode fusion | ✅ both bullets built; DoD borderline |
| **D** | Speculation inside continuous batching | ⚠️ composes and is lossless; headline claim unresolved |
| **E** | GPU execution | 🟡 **unblocked and started** — this document is the runbook |
| **F** | Production hardening | ✅ done, including AddressSanitizer |

**Test state:** 398 Python tests, 18 C++ cases, repo lint clean under pyflakes, 0 blocks leaked across a 180-cycle sanitizer run, 0 memory errors.

### The environment it runs on

```
CPU    Intel, 10 physical / 16 logical cores, 23.7 GB RAM
GPU    NVIDIA RTX 4050 Laptop, 6141 MiB VRAM, compute 8.9 (Ada)
torch  2.11.0+cu128     <- CUDA 12.8, GPU reachable
OS     Windows 11
```

**The GPU was the blocker for months, and it was never the hardware.** The installed torch was the CPU-only wheel, so `torch.cuda.is_available()` was False. The project documented this as "no CUDA device", which was simply untrue. Reinstalling torch against `cu128` unblocked everything. If you hit a claim in the older docs about hardware, distrust it.

---

## 3. Models — what to download, and why

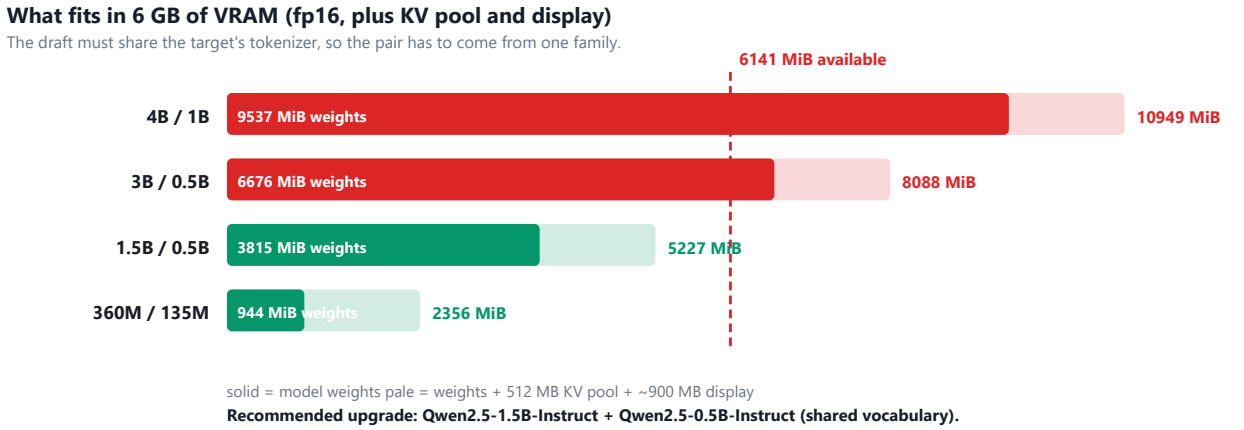



Figure 3 — VRAM budget on a 6 GB card. 4B/1B does not fit; the weights alone exceed the card.

The engine is model-agnostic but these are what the defaults and benchmarks assume. All are small on purpose: they fit, they load fast, and the project's claims are about *ratios*, which are portable, rather than absolute tok/s, which is not.

```
# Already cached if you have run anything (~1.7 GB total).
# Nothing to do — transformers pulls them on first use.
HuggingFaceTB/SmolLM2-360M-Instruct    # target model, the default
HuggingFaceTB/SmolLM2-135M-Instruct    # draft model, for speculation
```

Set them with `MEMFORGE_TARGET_MODEL` / `MEMFORGE_DRAFT_MODEL`, or `--model` / `--draft-model` on the benchmarks.

### If you want a bigger pair for Phase D

**The draft must share the target's tokenizer.** The engine takes one tokenizer and uses it for both, and rejection sampling indexes the target's distribution by the draft's token id — mismatched vocabularies would silently index the wrong token. So the pair must come from the same family.

VRAM budget on this 6 GB card, fp16, leaving room for the KV pool and display:

| pair | weights | total | verdict |
|---|---|---|---|
| 4B / 1B | 9537 MiB | 10949 MiB | **does not fit** |
| 3B / 0.5B | 6676 MiB | 8088 MiB | **does not fit** |
| **1.5B / 0.5B** | 3815 MiB | 5227 MiB | **fits** |
| 360M / 135M (current) | 944 MiB | 2356 MiB | fits |

So the realistic upgrade is `Qwen/Qwen2.5-1.5B-Instruct` + `Qwen/Qwen2.5-0.5B-Instruct` (shared 151936-token vocab, ~3 GB download). Be aware that is only a ~3× cost ratio against the current ~2.7×, so it is unlikely to flip speculation into a win — see §6.

### Hugging Face token (optional)

Not needed for cached models. If you pull new ones and hit rate limits, get a **Read** token from huggingface.co → Settings → Access Tokens, then:

```
huggingface-cli login          # stores it outside the repo
```

**Never put it in a file inside the repo.** There is no `.env` loading anywhere — `server/config.py` reads `os.environ` directly — so a key in a file would do nothing except eventually get committed.

---

## 4. First: prove the environment works

Run these three before trusting any number.

```
# 1. Is everything installed, and is CUDA really reachable?
python scripts/check_env.py

# 2. Does the whole thing still work? (~30 s)
python -m pytest tests/python -q --timeout=300

# 3. Every stage end to end: C++ tests, bindings, Python tests, benchmarks.
python scripts/run_pipeline.py
```

`run_pipeline.py` is the single "is the project healthy" command. It prints a table and **exits non-zero on failure** — it used to always exit 0, which silently invalidated every `EXIT=0` anyone had quoted, so trust the table.

> **Build the native extension first if `memforge_c` is missing:** `bash python setup.py build_ext --inplace --force` Use `--force` . setuptools does not track header dependencies, so a changed `.hpp` will not trigger a rebuild and you will test a stale `.pyd` . If the copy silently does nothing, an orphaned Python process is holding the file open — kill it first.

---

## 5. The benchmarks: what each one does, and the command

Every harness now takes `--device auto|cuda|cuda:N|cpu` and `--dtype auto|fp32|fp16|bf16`. Defaults are `auto`: CUDA if genuinely present, fp16 on GPU and fp32 on CPU.

**An explicit `--device cuda` on a machine without CUDA is an error, not a fallback.** A silent fallback would produce CPU numbers in a report labelled `cuda`, which is the worst failure mode available to a benchmark.

Results are written to `benchmarks/results/*.json`, each carrying the device, dtype, GPU name, VRAM and peak allocation.

### 5.1 Context sweep — the Phase A claim

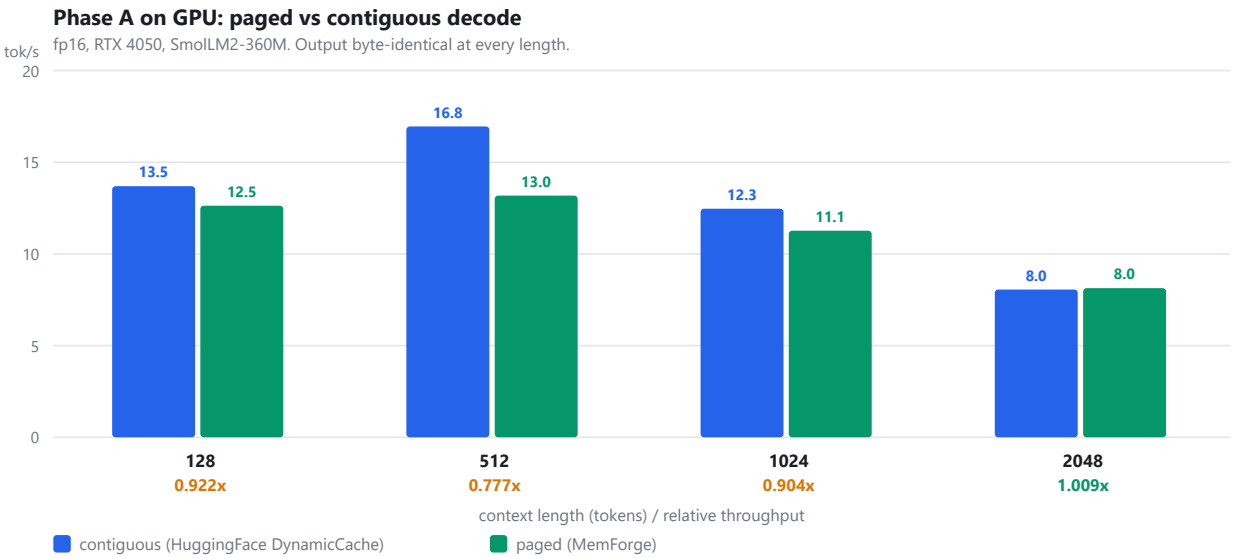



Figure 4 — Phase A on GPU. Paging loses at short contexts and catches up by 2048, where the target is met. Output byte-identical throughout.

```
python benchmarks/run_context_sweep.py --device cuda --contexts 128 512 1024 2048
```

**What it does:** decodes the same prompt three ways at each context length — stock HuggingFace `DynamicCache` (contiguous), MemForge's paged cache, and a batch of four — and compares throughput and output.

**Why it matters:** paging must not cost throughput. The DoD is "within 5% of contiguous at 2048 tokens".

**GPU result:**

| context | contiguous tok/s | paged tok/s | relative | identical |
|---|---|---|---|---|
| 128 | 13.54 | 12.49 | 0.92× | yes |
| 512 | 16.76 | 13.03 | 0.78× | yes |
| 1024 | 12.33 | 11.14 | 0.90× | yes |
| **2048** | **7.97** | **8.04** | **1.009×** | **yes** |

**Target MET on GPU**, and byte-identical at every length. Paging is slower at short contexts and that is the expected shape: per-step overhead is fixed while the benefit grows with context. The crossover is around 2048 here.

### 5.2 Prefix sharing — the Phase B claim

```
python benchmarks/run_prefix_sharing.py --device cuda --concurrency 1 4 --tokens 12 --prefix-tokens 256
```

**What it does:** sends N requests sharing a long preamble, with sharing off and then on, and compares blocks allocated and output.

**GPU result:** at N=4, **72 → 24 blocks (3.0× reduction)**, prefill work down to 0.27×, **output byte-identical**, 0 copy-on-write copies, 0 blocks leaked. At N=1 there is nothing to share and the ratio is exactly 1.000 — a useful sanity check that the measurement is not fooling itself.

### 5.3 Chunked prefill and fusion — the Phase C claim

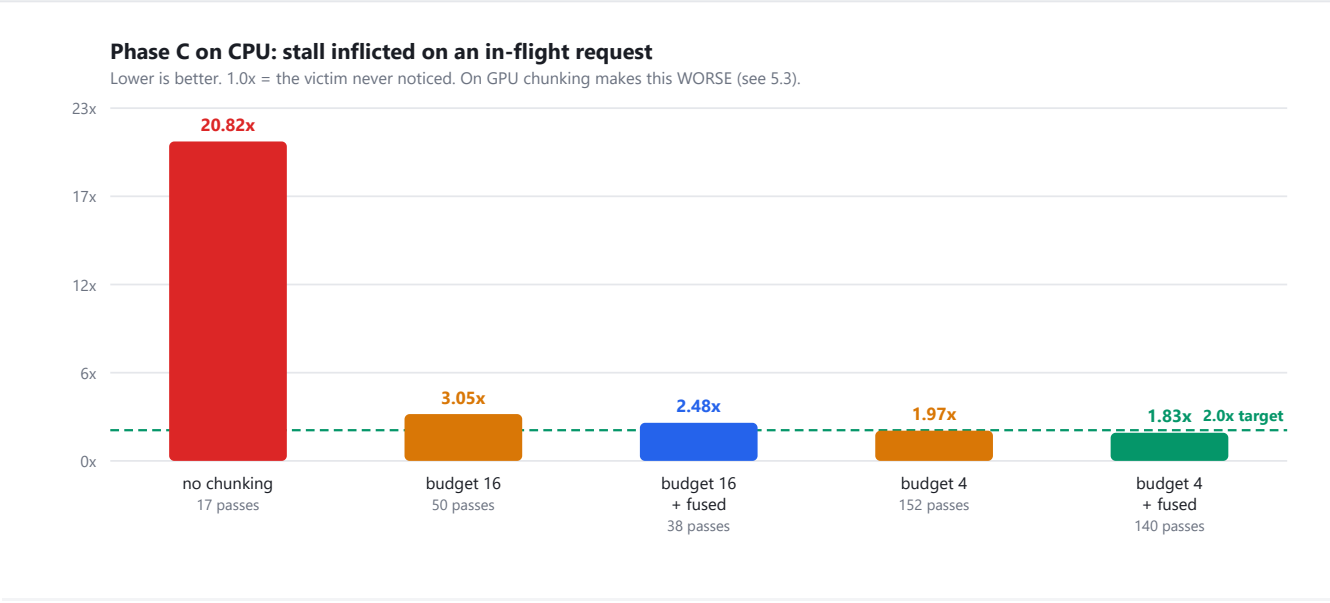



Figure 5 — Phase C on CPU. Each step toward the target costs more scheduler steps, visible in the pass counts.

```
# CPU is where this feature actually helps:
python benchmarks/run_chunked_prefill.py --prompt-tokens 512 --victim-tokens 16 --budgets 0 64 16 4 --fused
```

**What it does:** gets one request decoding steadily, then drops a long prompt on it, and measures the *victim's* inter-token gaps. A long prompt normally monopolises a step and every in-flight request skips a token.

**CPU result** (where prefill dominates):

| budget | fused | worst stall | passes |
|---|---|---|---|
| unlimited | no | 20.82× | 17 |
| 16 | no | 3.05× | 50 |
| 16 | yes | 2.48× | 38 |
| 4 | no | 1.97× | 152 |
| **4** | **yes** | **1.83×** | **140** |

**On GPU, chunking makes it worse** (baseline is already ~1.6–2.3× because a 512-token prefill is cheap there relative to a decode step, so extra scheduler steps cost more than the stall they avoid). The benchmark now says so plainly instead of reporting "target met" — it used to compare only against the 2.0× threshold and never against the unchunked baseline.

**Read this as: chunking is a latency tool for the regime where prefill dominates.** That is CPU, or long prompts, or big models — not a small model on a GPU.

### 5.4 Speculation inside batching — the Phase D claim

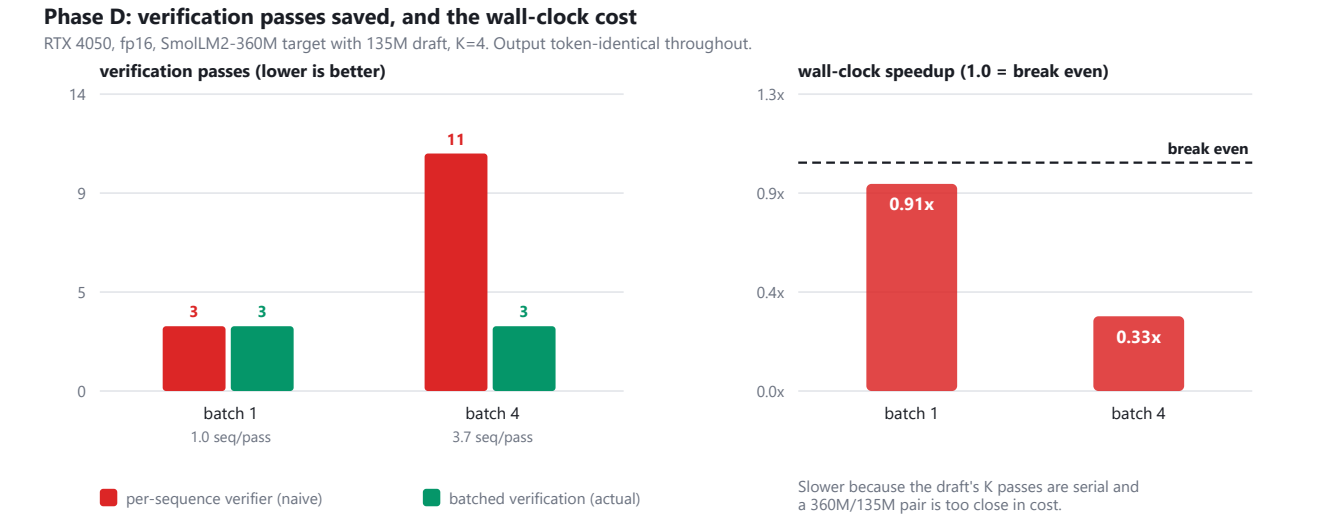



Figure 6 — Phase D, both halves. The structural win is real (left); the wall clock is still a loss (right).

```
python benchmarks/run_speculative_batching.py --device cuda --batch-sizes 1 2 4 --max-new-tokens 16
python benchmarks/run_speculative_batching.py --device cuda --self-draft   # forces 100% acceptance
```

**What it does:** runs the same prompts with and without speculation, checks the tokens are identical, and counts verification passes both as batched (actual) and per-sequence (what a naive implementation would cost).

**Result — read the two halves separately:**

*The composition works.* At batch 4, verification costs **5 target passes where a per-sequence verifier needs 15** — 3.0 sequences per pass. Output per target pass rises from 3.37 to 4.92 tokens. Output is token-identical at every batch size. Before this, speculating over N sequences cost N target passes per round and was strictly worse than not speculating; that obstacle is gone.

*It is still not worth enabling here.* **0.62–0.77× on CPU, 0.33–0.91× on GPU** — slower, and worse on GPU. That direction is the opposite of the naive expectation and it is worth understanding: a single target pass on a GPU is latency-bound rather than compute-bound, so it is *already cheap*, and the draft's K serial passes dominate a larger share of each round. **Speculation needs an expensive target, not a fast device.**

### 5.5 Stress and soak

```
python benchmarks/run_stress_test.py                    # 110 requests at concurrency 8
python scripts/traffic_simulator.py --concurrency 6 --requests 400 \
       --long-prompt-rate 0.6 --disconnect-rate 0.1     # needs a running server
```

**What they do:** sustained load with mid-stream disconnects, sampling RSS and block accounting. The claim is flat memory and zero leaked blocks. (The stress harness once measured the *launcher* process — 5.6 MB — instead of the child doing the work — 1002.7 MB. It sums the process tree now.)

### 5.6 Memory safety (C++)

```
python scripts/cpp_asan.py                    # full run
python scripts/cpp_asan.py --verify-tooling   # just: what can the tools detect?
```

**What it does:** hammers the allocator, prefix cache and scheduler through 180 cycles of concurrent requests under a sanitizer, asserting every cycle that the free list returns to its starting size.

**Platform-dependent, and the difference matters:** - **Linux/macOS** — one pass. gcc and clang ship LeakSanitizer with ASan, so leaks, use-after-free and overflows are all covered. - **Windows** — two passes, because **MSVC's ASan has no leak detection**. The second builds against the debug CRT with `_CRTDBG_LEAK_CHECK_DF`.

`--verify-tooling` compiles a program with a known leak, a known use-after-free and a known overflow and reports which pass caught which. **Run it first.** A misconfigured sanitizer detects nothing, which is indistinguishable from clean code — this check has already caught two faults that read as success.

### 5.7 Live server end to end

```
# terminal 1
MEMFORGE_MAX_PREFILL_TOKENS_PER_STEP=64 MEMFORGE_MAX_NUM_SEQS=8 \
  python -m uvicorn server.app:app --port 8000

# terminal 2
cd frontend && npm install && npm run dev      # http://localhost:3000

# terminal 3 — give the dashboard something to show
python scripts/traffic_simulator.py --concurrency 6 --requests 400 --long-prompt-rate 0.6
```

Every dashboard panel has a **Learn more** button explaining what it measures and how. Live on GPU with that traffic: 3.58 tokens/forward pass, 81.75% prefix hit rate, 2065 MB of 6140 MB VRAM, 0 preemptions.

**Use `/v1/chat/completions`, not `/v1/completions`, for realistic traffic.** The raw completions path does not apply the chat template, so an instruct model emits EOS almost immediately and you will see 1-token responses and a dead dashboard.

### Feature flags worth knowing

| env var | what it does | default |
|---|---|---|
| `MEMFORGE_MAX_PREFILL_TOKENS_PER_STEP` | chunked prefill budget; 0 = off | 0 |
| `MEMFORGE_FUSED_PREFILL` | chunk + decode in one pass (needs chunking) | off |
| `MEMFORGE_BATCHED_SPECULATION` | speculation inside the batch loop | off |
| `MEMFORGE_BLOCK_ATTENTION` | block-wise attention, O(block) working memory | off |
| `MEMFORGE_DECODE_VIEW_CACHE` | contiguous mirror per sequence, faster, costs memory | off |
| `MEMFORGE_LOOKAHEAD_K` | speculative lookahead | 4 |

**Environment variables are read at import time.** Setting one after the server starts does nothing. Everything optional is off by default because each one is a trade, and several are *losses* on this hardware.

---

## 6. What is actually left, and how to finish it

### 6.1 Phase E — record GPU numbers (mostly done, needs completion)

Four of six harnesses have been run on GPU and their JSON updated. Remaining:

```
python benchmarks/run_live_engine.py --device cuda      # still has a CPU-only result on file
python benchmarks/run_stress_test.py                    # add device reporting
```

Then put both sets of numbers in the README — that is the literal DoD.

**Expect to find bugs.** Running on GPU for the first time surfaced three real ones in a single afternoon: - input tensors built on CPU, fed to a CUDA model - `ModelKVSpec` hardcoded to `(float32, "cpu")`, so the pool landed on the wrong device - a verdict string that printed "on this CPU" while measuring a GPU

**Where to look for more:** anything that hardcodes a dtype or device, anything calling `.item()` / `int(tensor)` / `.tolist()` in a hot loop (each forces a GPU sync), and **long contexts in fp16** — accumulation error is the one thing nothing has exercised, because the fp16 checks so far used a tiny model at short contexts.

## 6.2 The real optimisation target nobody has touched

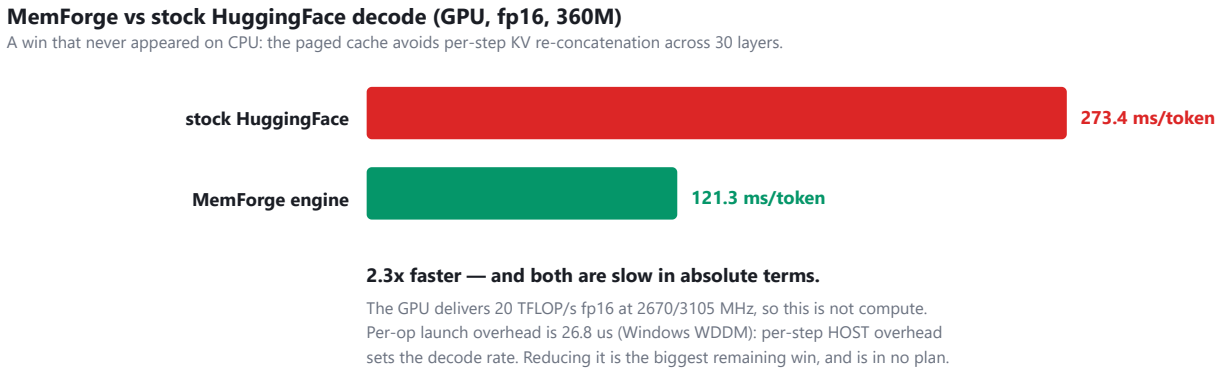



Figure 7 — The unmeasured win, and the overhead that is the actual bottleneck.

Measured on this machine:

```
plain HuggingFace decode, fp16, GPU   273.4 ms/token
MemForge engine,          fp16, GPU   121.3 ms/token    (2.3x faster)
```

MemForge beating stock HF by 2.3× is a genuine, previously-unmeasured win — the paged cache avoids HF's per-step KV re-concatenation across 30 layers, and that advantage did not show up on CPU at all. **Put this in the README.**

But **121 ms/token for a 360M model on this GPU is poor.** The GPU is healthy (20 TFLOP/s fp16, clocks 2670/3105 MHz), so the time is not compute. Per-op launch overhead measured **26.8 µs** — high, and typical of Windows WDDM. With ~30 layers and many kernels each, launch overhead plus per-step Python work sets the decode rate.

So the highest-value remaining work is **reducing per-step host overhead**: fewer, larger kernel launches; eliminating host-device syncs on the hot path; and CUDA graphs for the decode step. None of this is in any phase plan. It is the thing that would actually make the engine fast.

## 6.3 Phase D — resolve or reframe the headline claim

The plan says "a GPU run either confirms or refutes the 2-3× claim". It cannot be confirmed here: the claim is about an 8B-class target and **6 GB will not hold one in fp16**.

Two honest options: 1. **Reframe the DoD** to the pair that fits (1.5B/0.5B) and report whatever it gives. Likely still below 1.0×, which is a legitimate finding. 2. **Declare it out of scope on this hardware** and record the structural result that *was* achieved — batched verification, 3 sequences per target pass, lossless — as the phase's actual deliverable.

I would take (1), measure it, and write down the number whatever it is.

## 6.4 Phase C — reframe a borderline DoD

The bar is "p99 inter-token latency unaffected". At budget 4 with fusion the worst stall is **1.83–2.24× across four repeat runs** against a 2.0× threshold, so runs land on either side. The structural cause is gone; what remains is the chunk's own token count, which cannot vanish.

Either accept ~2× as "effectively unaffected" and say so, or restate the bar in terms of what was actually fixed. Carrying an unmeetable criterion is worse than restating it.

---

## 7. Things that will waste your time if you do not know them

1. `--force` **on** `build_ext`**.** setuptools does not track header deps. Without it you test a stale `.pyd`. If the copy silently no-ops, an orphaned Python process holds the file open.
2. `pkill` **does not stop uvicorn on Windows.** Use `Get-NetTCPConnection` / `taskkill`.
3. `pytest --timeout=300`**, always.** A mutation run once took **6h38m** and reported "45 passed" instead of failing. `pytest-timeout` is installed; use it.
4. **The C++ test runner has four build files.** Adding a `.cpp` to the engine means adding it to `scripts/cpp_test_runner.py`'s source lists too, or you get an LNK2019 that looks like a code error.
5. **A tiny random model is a weak test oracle.** Comparing greedy tokens over a 256-token vocabulary misses a lot — wrong RoPE positions and a never- advancing cache cursor all passed. Prefer logit-level comparison, exact invariant assertions, and self-speculation (target as its own draft) to force full acceptance.
6. `cached_prefix_tokens` **is not a sharing metric.** It counts chunked-prefill resumption too. Use `prefix_cache_hits`.
7. **Prefix sharing needs the prompts to overlap in time.** A finished sequence's blocks are freed and evicted, so running one prompt to completion before submitting its twin shares nothing.

---

## 8. One-command summary

```
python setup.py build_ext --inplace --force     # if memforge_c is missing
python scripts/check_env.py                     # is CUDA really there?
python -m pytest tests/python -q --timeout=300  # 398 tests
python scripts/run_pipeline.py                  # everything, with a table
python scripts/cpp_asan.py --verify-tooling     # can the sanitizers see anything?
```

If all five are green, the project is healthy and you can trust what the benchmarks tell you. If you add a number to this document, say what hardware produced it and whether it was favourable.